

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

*I **Deepa preya H / 192421047** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

1. smart city surveillance system

Introduction

A smart city surveillance system continuously monitors public places using multiple cameras. The captured video data is transferred from the I/O devices (cameras) to the main memory, where the CPU processes each video frame to detect suspicious activities. During peak hours, many cameras transmit high-resolution video simultaneously, resulting in heavy data traffic. This increases the workload on the CPU, memory, and system bus, causing delays in processing and threat detection.

Interaction between CPU, Memory and I/O

The CPU, memory, and I/O devices work together to process surveillance data efficiently.

- **CPU (Central Processing Unit):** The CPU fetches instructions from memory, decodes them, executes image processing algorithms, and generates threat detection results. When multiple video streams arrive simultaneously, the CPU becomes overloaded, increasing processing time.
- **Memory (RAM):** RAM temporarily stores incoming video frames and program instructions. During peak hours, continuous video streams occupy large memory space, increasing memory access time. The CPU must wait longer to retrieve data, reducing overall performance.
- **I/O Devices:** Surveillance cameras act as input devices that continuously send video data to the system. If many cameras transmit data simultaneously, the I/O subsystem becomes congested, delaying data transfer to memory.

The interaction follows this sequence:

Camera → I/O Controller → System Bus → RAM → CPU → Threat Detection → Display/Alarm

If any component becomes slow, the entire system experiences lag and delayed response.

Impact of Bus Structures

Single-Bus Architecture

In a single-bus architecture, the CPU, memory, and I/O devices share one common communication bus. Since only one device can use the bus at a time, multiple devices must wait for bus access. During peak hours, simultaneous data transfer from several cameras creates bus contention, increasing waiting time and reducing system throughput.

Advantages:

- Simple architecture
- Low implementation cost
- Easy to maintain

Disadvantages:

- High bus congestion
- Increased CPU waiting time
- Poor performance under heavy workloads

Multi-Bus Architecture

In a multi-bus architecture, separate buses are provided for communication between the CPU, memory, and I/O devices. This allows multiple data transfers to occur simultaneously without interference.

Advantages:

- Parallel data transfer
- Reduced bus contention
- Faster communication
- Better CPU utilization
- Higher system throughput

Disadvantages:

- Higher hardware cost
- More complex design

Instruction Execution Cycle

The CPU processes every instruction through four stages:

1. **Fetch** – Retrieves the instruction from memory.
2. **Decode** – Interprets the instruction.
3. **Execute** – Performs the required operation.
4. **Store (Write Back)** – Saves the result in memory.

When millions of instructions are executed every second, delays may occur during memory access or instruction fetching, increasing the execution time and reducing system performance.

Methods to Improve System Performance

The following architectural improvements can enhance system performance:

- **Instruction Pipelining:** Executes multiple instructions simultaneously by overlapping different stages of the instruction cycle, improving throughput.
- **Cache Memory:** Frequently used instructions and data are stored in cache, reducing memory access time.
- **Direct Memory Access (DMA):** Enables I/O devices to transfer data directly to memory without continuous CPU involvement, reducing processor workload.
- **Multi-Core Processor:** Different processor cores can handle multiple video streams simultaneously, improving parallel processing.
- **Branch Prediction and Prefetching:** These techniques reduce execution delays by predicting upcoming instructions and loading them in advance.

Proposed Solution

To improve the surveillance system, the organization should replace the existing single-bus architecture with a multi-bus architecture to reduce communication delays. DMA can be used for efficient data transfer from cameras to memory, while cache memory and pipelining can reduce instruction execution time. Multi-core processors should be employed to process multiple video streams in parallel. These improvements reduce CPU waiting time, lower system latency, and enable faster threat detection.

Conclusion

The lag in the smart city surveillance system is mainly caused by heavy interaction between the CPU, memory, and I/O devices during peak traffic conditions. A single-bus architecture creates communication bottlenecks, while a multi-bus architecture allows simultaneous data transfers and improves overall efficiency. By implementing architectural enhancements such as instruction pipelining, cache memory, DMA, and multi-core processors, the surveillance system can achieve faster processing, reduced delays, improved throughput, and accurate real-time threat detection.

3. An embedded system based on an 8085 microprocessor

Introduction

An **embedded system** is a computer system designed to perform a specific task efficiently. In an industrial temperature control unit, the **8085 microprocessor** receives temperature data from sensors, processes the information, and controls actuators such as heaters or cooling fans. The accuracy of the system depends on the proper use of **addressing modes** and the correct execution of instructions. If the wrong addressing mode is used, incorrect data may be processed, leading to inaccurate temperature readings and improper system operation.

Understanding the Problem

In the temperature control system:

- Sensors continuously measure the temperature.
- The sensor data is sent to the **8085 microprocessor**.
- The processor reads the data, compares it with the required temperature, and sends control signals to the actuators.

- If the program uses incorrect addressing modes, the processor may read data from the wrong memory location or register, resulting in incorrect temperature values and faulty control actions.

Addressing Modes in 8085

Addressing modes specify how the processor accesses data or operands. The main addressing modes in the **8085 microprocessor** are:

1. Immediate Addressing Mode

The data is provided directly in the instruction.

Example:

MVI A, 25H

This instruction loads the value **25H** directly into register A.

Application: Used for fixed values such as threshold temperature limits.

2. Register Addressing Mode

The operand is stored in a CPU register.

Example:

MOV A, B

This instruction copies the contents of register B into register A.

Application: Used for fast data transfer between registers.

3. Direct Addressing Mode

The memory address of the data is specified in the instruction.

Example:

LDA 2500H

This instruction loads the data stored at memory location **2500H** into register A.

Application: Used to read sensor values stored in memory.

4. Register Indirect Addressing Mode

The memory address is stored in a register pair (HL).

Example:

MOV A, M

This instruction transfers the data from the memory location pointed to by the HL register pair into register A.

Application: Used for accessing sensor data stored at different memory locations.

5. Implied Addressing Mode

The operand is implied by the instruction itself.

Example:

CMA

This instruction complements the contents of the accumulator.

Application: Used for operations that do not require an explicit operand.

Possible Causes of Incorrect Temperature Readings

Incorrect temperature readings may occur due to:

- Using the wrong addressing mode in the program.
- Reading data from an incorrect memory location.
- Incorrect initialization of registers.
- Faulty sensor data transfer.
- Memory corruption due to programming errors.
- Incorrect use of instructions for data transfer or comparison.

These errors may cause the processor to process invalid temperature values, leading to incorrect operation of heaters or cooling systems.

Effective Use of the 8085 Instruction Set

The following instructions help ensure reliable operation:

- **LDA** – Reads sensor data from memory.
- **STA** – Stores processed data in memory.
- **MOV** – Transfers data between registers.
- **MVI** – Loads constant values into registers.
- **CMP** – Compares the current temperature with the reference value.
- **JZ, JC, JNZ** – Performs decision-making based on comparison results.
- **IN** – Reads input from sensor ports.
- **OUT** – Sends control signals to actuators.

By selecting the appropriate instruction and addressing mode, the processor can accurately process sensor data and control industrial equipment.

Suggested Solution

To improve the accuracy of the temperature control system:

- Use the correct addressing mode for accessing sensor data.
- Store sensor values in predefined memory locations.
- Initialize all registers properly before program execution.
- Use instructions such as **LDA, MOV, CMP, IN, and OUT** correctly.
- Validate sensor readings before processing.
- Test the program thoroughly to eliminate addressing errors.

Conclusion

The performance of an **8085-based temperature control system** depends on the correct use of addressing modes and instructions. Improper addressing can result in incorrect temperature readings and unreliable system operation. By selecting appropriate addressing modes, using the 8085 instruction set effectively, and validating sensor data, the embedded system can achieve accurate processing, reliable control, and efficient industrial automation.

5. A data communication system

Introduction

In modern data communication systems, packet information is commonly represented in hexadecimal (Hex) format because it is compact, easy to read, and simplifies debugging. However, the CPU internally processes all data in binary (0s and 1s). Therefore, accurate conversion between hexadecimal and binary is essential for correct data transmission and instruction execution. Any mistake in the conversion process can result in incorrect packet interpretation, processing errors, and system failures.

Understanding the Problem

In the given scenario:

- Packet data is stored and transmitted in hexadecimal format.
- The software engineer incorrectly converts hexadecimal values into binary.
- The CPU processes incorrect binary data.
- This leads to incorrect instruction execution, wrong packet interpretation, and communication errors.

Therefore, proper number system conversion is essential for reliable data communication.

Importance of Hexadecimal and Binary Conversion

The hexadecimal number system uses 16 symbols (0–9 and A–F), while the binary number system uses only 0 and 1.

Each hexadecimal digit represents 4 binary bits, making conversion simple and efficient.

Example

Hexadecimal: 3A

Binary: 0011 1010

The CPU converts hexadecimal values into binary before processing instructions.

Advantages of Using Hexadecimal

- Easy to read and write.
- Represents large binary numbers in a shorter form.
- Simplifies memory addressing and debugging.
- Reduces the possibility of reading errors.

Effects of Incorrect Conversion

Incorrect conversion between hexadecimal and binary can cause several problems:

1. Incorrect Instruction Execution

If a hexadecimal value is converted incorrectly, the CPU interprets it as a different binary instruction, leading to incorrect execution.

2. Data Corruption

Wrong binary values may alter packet contents, resulting in corrupted or incomplete data transmission.

3. Communication Errors

Incorrect packet headers or addresses may cause packets to be sent to the wrong destination or rejected by the receiver.

4. Reduced System Reliability

Repeated conversion errors can decrease the accuracy and reliability of real-time communication systems.

Role of CPU Design

An efficient CPU design helps reduce processing errors and improves overall system performance.

1. Fast Instruction Execution

A high-performance CPU executes instructions quickly, reducing delays in packet processing.

2. Efficient Cache Memory

Cache memory stores frequently used data, reducing memory access time and improving execution speed.

3. Instruction Pipelining

Pipelining allows multiple instructions to be processed simultaneously, increasing throughput and reducing execution time.

4. Error Detection Mechanisms

Modern CPUs include error-checking techniques that help identify invalid instructions and incorrect data before execution.

Role of Benchmarking

Benchmarking is the process of measuring and evaluating the performance of a computer system.

It helps to:

- Measure CPU performance.
- Compare execution speed.
- Detect processing errors.
- Identify memory bottlenecks.
- Evaluate system efficiency under different workloads.

By performing regular benchmarking, engineers can identify conversion-related errors and improve system performance.

Suggested Solution

To minimize conversion-related errors in real-time communication systems, the following measures should be adopted:

- Use reliable hexadecimal-to-binary conversion algorithms.
- Validate packet data before execution.
- Implement error detection techniques such as parity checking or CRC.

- Use CPUs with cache memory and instruction pipelining for faster execution.
- Perform regular benchmarking to monitor system performance and identify bottlenecks.
- Thoroughly test software to eliminate conversion and processing errors.

These measures improve data accuracy, reduce processing errors, and enhance overall system reliability.

Conclusion

Hexadecimal and binary number systems play a vital role in data communication systems. Accurate conversion between these number systems ensures correct instruction execution and reliable packet processing. Errors in conversion can lead to incorrect data interpretation, communication failures, and reduced system performance. By using efficient CPU architecture, implementing error detection mechanisms, and conducting regular benchmarking, real-time systems can achieve faster processing, improved accuracy, and reliable communication.